

Homework 6

Haozheng Wang

December 2025

1 Data augmentation

5 ways are chosen to augment our data: RandomCrop, RandomHorizontalFlip, RandomRotation, RandomAffine and ColorJitter, as is shown in the following code block.

```
1 transform_train = transforms.Compose([
2     transforms.RandomCrop(32, padding=4),
3     transforms.RandomHorizontalFlip(p=0.5),
4     transforms.RandomRotation(15),
5     transforms.RandomAffine(degrees=0, translate=(0.1, 0.1),
6         scale=(0.9, 1.1), shear=5),
7     transforms.ColorJitter(brightness=0.2, contrast=0.2,
8         saturation=0.2, hue=0.1),
9     transforms.ToTensor(),
10    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
11])
```

2 Implement VGG for image classification

We first implement our own function *conv_block* to create a "sandwich" convolution block, which is consisted of a convolution layer, a batch-normalization layer and a ReLU activation function. It should be noted that dropout is not adopted here, and it was not abandoned for no reason. Empirically, adding a dropout layer after each convolution layer is prone to cause training unstabilities and difficulties, probably because that dropout disturbs the process of feature extracting instead of enhancing it. However, we add a dropout after each FC layer in our full-connected classifier, which will be shown shortly afterwards.

```
1 def conv_block(self, in_channels: int, out_channels: int):
2     return nn.Sequential(
3         nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
4         nn.BatchNorm2d(out_channels),
5         nn.ReLU(inplace=True))
```

The overall structure of our VGG generally follows the design of VGG-16, and the only difference is that the size of fully connected layers in our classifier is dwindled to reduce the amount of parameters. For the complete implementation of our VGG, please refer to **model.py**.

```

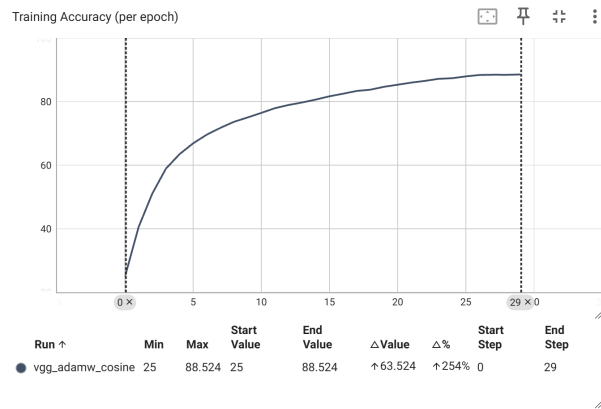
1 # 1. define multiple convolution and downsampling layers
2 self.features = nn.Sequential(
3     # Stage 1: conv-conv-pool
4     self.conv_block(3, 64),
5     self.conv_block(64, 64),
6     nn.MaxPool2d(2, 2), # -> N * 64 * 16 * 16
7     # Stage 2: conv-conv-pool
8     self.conv_block(64, 128),
9     self.conv_block(128, 128),
10    nn.MaxPool2d(2, 2), # -> N * 128 * 8 * 8
11    # Stage 3: conv-conv-pool
12    self.conv_block(128, 256),
13    self.conv_block(256, 256),
14    nn.MaxPool2d(2, 2), # -> N * 256 * 4 * 4
15    # Stage 4: conv-conv-conv-pool
16    self.conv_block(256, 512),
17    self.conv_block(512, 512),
18    self.conv_block(512, 512),
19    nn.MaxPool2d(2, 2), # -> N * 512 * 2 * 2
20    # Stage 5: conv-conv-conv-pool
21    self.conv_block(512, 512),
22    self.conv_block(512, 512),
23    self.conv_block(512, 512),
24    nn.MaxPool2d(2, 2), # -> N * 512 * 1 * 1
25 )
26 # 2. define full-connected layer to classify
27 self.classifier = nn.Sequential(
28     nn.Linear(512, 512),
29     nn.BatchNorm1d(512),
30     nn.ReLU(inplace=True),
31     nn.Dropout(),
32     nn.Linear(512, 256),
33     nn.BatchNorm1d(256),
34     nn.ReLU(inplace=True),
35     nn.Dropout(),
36     nn.Linear(256, 10)
37 )

```

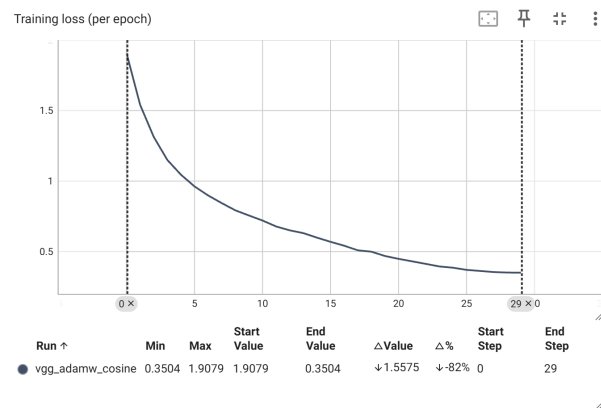
The configurations of our experiment on VGG:

- optimizer: AdamW (AdamW_lr=0.001)
- scheduler: CosineAnnealingLR (CosineAnnealingLR_T_max=30)
- epoch_num=30
- batch_size=128

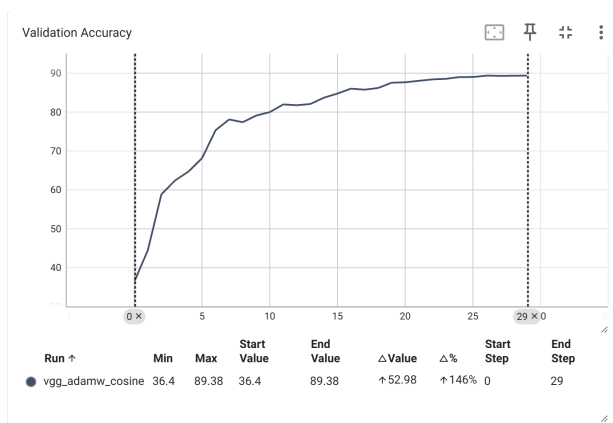
The test accuracy of our VGG is 89.38%. Success!



(a) Training Accuracy



(b) Training Loss



(c) Validation Accuracy

Figure 1: VGG

3 Implement ResNet for image classification

For this section, our implementation gives great credit to Dive into Deep Learning.

First, we implement basic residual block in preparation for the implementation of the entire ResNet. The following figure¹ clearly shows what a ResBlock looks like in detail: downsampling is conducted in the first weight layer (i.e., invoke parameter *stride* in *Conv2*) at the same time as the number of channels is doubled, and these operations are not performed in the second weight layer. It also inspire us on how to design the 1×1 Conv layer, as we have known that downsampling and doubling in_channel ought to be conducted so that the input can be reshaped in alignment with the output of our residual block.

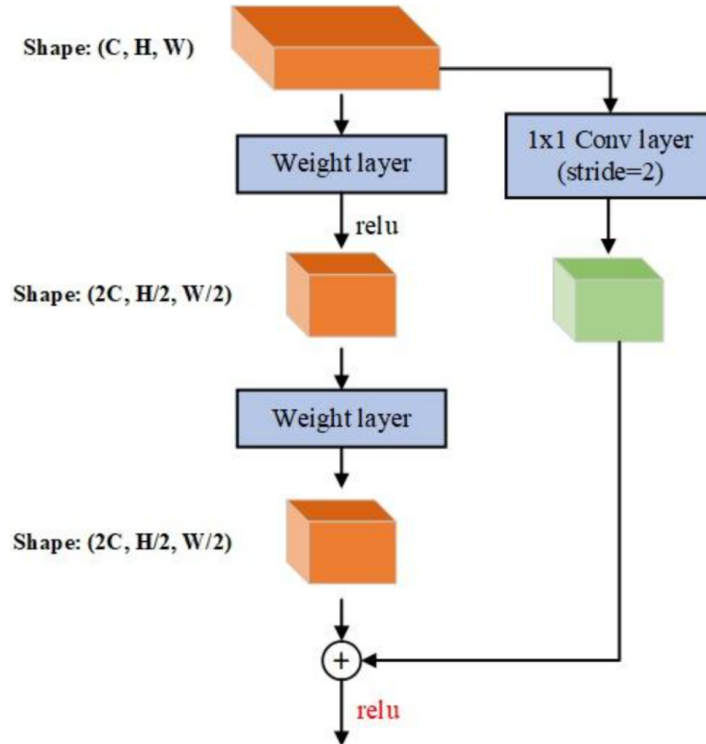


Figure 2: ResBlock

```
1 class ResBlock(nn.Module):
2     def __init__(self, in_channel, out_channel, stride):
3         super().__init__()
4         self.in_channel = in_channel
5         self.out_channel = out_channel
6         self.stride = stride
```

¹<https://zhuanlan.zhihu.com/p/514752578>

```

7      # 1. define double convolution
8      self.conv1 = nn.Conv2d(in_channel, out_channel, kernel_size
9                               =3, stride=stride, padding=1)
10     self.bn1 = nn.BatchNorm2d(out_channel)
11     self.relu1 = nn.ReLU(inplace=True)
12     self.conv2 = nn.Conv2d(out_channel, out_channel,
13                             kernel_size=3, stride=1, padding=1)
14     self.bn2 = nn.BatchNorm2d(out_channel)
15     self.relu2 = nn.ReLU(inplace=True)
16     # 2. if in_channel != out_channel or stride != 1, deifine 1
17     #     x1 convolution layer to change the channel or size.
18     self.conv3 = None
19     if in_channel != out_channel or stride != 1:
20         self.conv3 = nn.Conv2d(in_channel, out_channel,
21                                 kernel_size=1, stride=stride)
22
23     def forward(self, x: torch.Tensor):
24         # 1. convolve the input
25         out = self.bn2(self.conv2(self.relu1(self.bn1(self.conv1(x)
26                                                     )))))
27         # 2. if in_channel != out_channel or stride != 1, change
28         #     the channel or size of 'x' using 1x1 convolution.
29         if self.in_channel != self.out_channel or self.stride != 1:
30             x = self.conv3(x)
31         # 3. Add the output of the convolution and the original
32         #     data (or from 2.)
33         out += x
34         # 4. relu
35         out = self.relu2(out)
36         return out

```

The implementation of our ResNet is shown below. The main body of ResNet uses four modules composed of residual blocks, each module using two residual blocks with the same number of output channels. The number of channels in the first module is the same as the number of input channels. Each subsequent module doubles the number of channels from the previous module in the first residual block and halves the height and width. Note that we have made special adjustments to the first module.

```

1  class ResNet(nn.Module):
2      def __init__(self):
3          super().__init__()
4          # 1. define convolution layer to process raw RGB image
5          self.conv = nn.Sequential(
6              nn.Conv2d(in_channels=3, out_channels=64, kernel_size
7                          =3, padding=1),
8              nn.BatchNorm2d(num_features=64),
9              nn.ReLU(inplace=True),
10             nn.MaxPool2d(kernel_size=2, stride=2)
11         )
12         # 2. define multiple residual blocks
13         self.resnet_block_1 = nn.Sequential(*self.resnet_block(64,
14                                                                     64,
15                                                                     2,
16                                                                     first_block=True))
17         self.resnet_block_2 = nn.Sequential(*self.resnet_block(64,
18                                                                     128,
19                                                                     2))

```

```

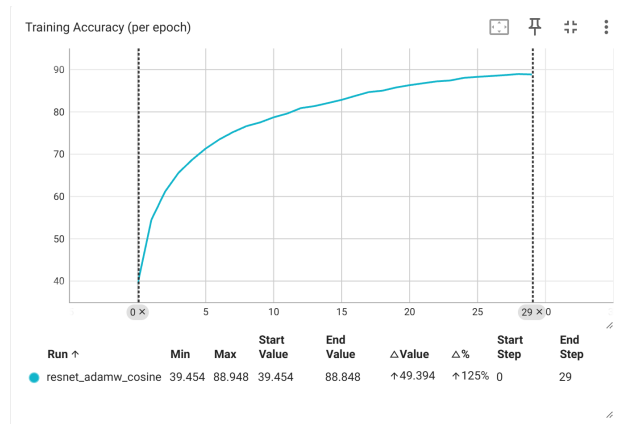
14         self.resnet_block_3 = nn.Sequential(*self.resnet_block(128,
15         256, 2))
16         self.resnet_block_4 = nn.Sequential(*self.resnet_block(256,
17         512, 2))
18         # 3. define full-connected layer to classify
19         self.global_avg_pool = nn.AdaptiveAvgPool2d((1, 1))
20         self.fc = nn.Linear(512, 10)
21
22     def resnet_block(self, in_channel, out_channel, num_ResBlocks,
23     first_block=False):
24         resnet_block = []
25         for i in range(num_ResBlocks):
26             if i == 0 and not first_block:
27                 resnet_block.append(ResBlock(in_channel,
28                 out_channel, stride=2))
29             else:
30                 resnet_block.append(ResBlock(out_channel,
31                 out_channel, stride=1))
32         return resnet_block
33
34     def forward(self, x: torch.Tensor):
35         # extract features
36         x = self.conv(x)
37         x = self.resnet_block_4(self.resnet_block_3(self.
38         resnet_block_2(self.resnet_block_1(x))))
39         # classification
40         x = self.global_avg_pool(x)
41         x = x.view(x.size(0), -1)
42         out = self.fc(x)
43         return out

```

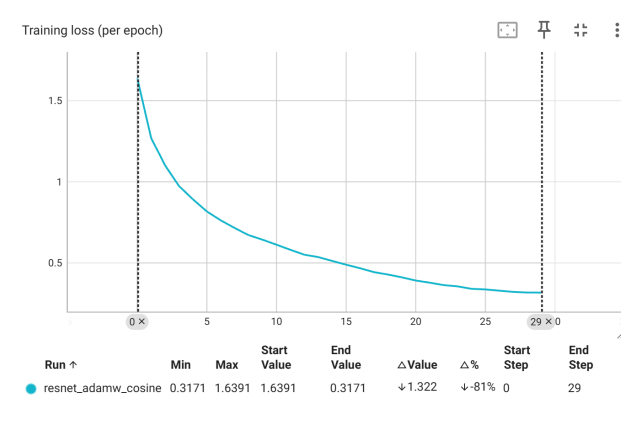
The configurations of our experiment on ResNet:

- optimizer: AdamW (AdamW_lr=0.001)
- scheduler: CosineAnnealingLR (CosineAnnealingLR_T_max=30)
- epoch_num=30
- batch_size=128

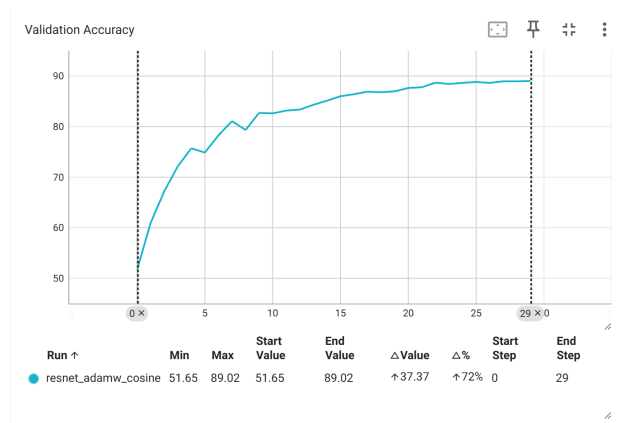
The test accuracy of our ResNet is 89.02%. Success!



(a) Training Accuracy



(b) Training Loss



(c) Validation Accuracy

Figure 3: ResNet

4 Implement ResNeXt for image classification

We first implement bottleneck residual block in preparation for the implementation of the entire ResNext. Grouped convolutions are carried out in the second convolution block (at the same time as the downsampling), which is the absolute soul of ResNeXt.

```
1 class ResNextBlock(nn.Module):
2     def __init__(self, in_channel, out_channel, bottle_neck, group,
3         stride):
4         super().__init__()
5         hidden_channel = out_channel // bottle_neck
6         self.in_channel = in_channel
7         self.out_channel = out_channel
8         self.stride = stride
9         # 1. define convolution
10        self.conv1 = nn.Conv2d(in_channel, hidden_channel,
11            kernel_size=1)
12        self.bn1 = nn.BatchNorm2d(hidden_channel)
13        self.relu1 = nn.ReLU(inplace=True)
14        self.conv2 = nn.Conv2d(hidden_channel, hidden_channel,
15            kernel_size=3, stride=stride, padding=1, groups=group)
16        self.bn2 = nn.BatchNorm2d(hidden_channel)
17        self.relu2 = nn.ReLU(inplace=True)
18        self.conv3 = nn.Conv2d(hidden_channel, out_channel,
19            kernel_size=1)
20        self.bn3 = nn.BatchNorm2d(out_channel)
21        self.relu3 = nn.ReLU(inplace=True)
22        # 2. if in_channel != out_channel or stride != 1, define 1
23        #    x1 convolution layer to change the channel or size.
24        self.conv4 = None
25        if in_channel != out_channel or stride != 1:
26            self.conv4 = nn.Conv2d(in_channel, out_channel,
27                kernel_size=1, stride=stride)
28
29    def forward(self, x: torch.Tensor):
30        # x: input image, shape: [B * C * H* W]
31        # 1. convolve the input.
32        out = self.relu1(self.bn1(self.conv1(x)))
33        out = self.relu2(self.bn2(self.conv2(out)))
34        out = self.bn3(self.conv3(out))
35        # 2. if in_channel != out_channel or stride != 1, change
36        #    the channel or size of 'x' using 1x1 convolution
37        # 3. Add the output of the convolution and the original
38        #    data (or from 2.)
39        if self.in_channel != self.out_channel or self.stride != 1:
40            out += self.conv4(x)
41        else:
42            out += x
43        # 4. relu
44        out = self.relu3(out)
45        return out
```

To implement ResNeXt more conveniently, we create our own function *resnext_layer* in exactly the same way as we have down in completing ResNet.

```

1 def resnext_layer(self, in_channel, out_channel, num_blocks,
2   bottle_neck, group, first_block=False):
3     resnext_layer = []
4     for i in range(num_blocks):
5         if i == 0 and not first_block:
6             resnext_layer.append(ResNextBlock(in_channel,
7               out_channel, bottle_neck, group, stride=2))
8         else:
9             resnext_layer.append(ResNextBlock(out_channel,
10               out_channel, bottle_neck, group, stride=1))
11     return resnext_layer

```

We set `bottle_neck=2` and `group=32` for residual blocks in our ResNeXt. The remaining parts of this network almost stay the same as ResNet, and you can refer to **models.py** for more details.

```

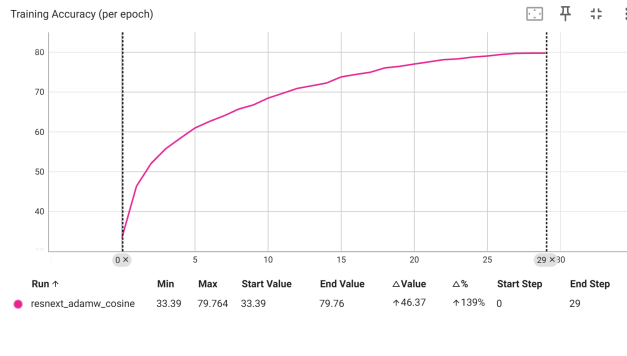
1 # define multiple residual blocks
2 self.resnext_layer_1 = nn.Sequential(*self.resnext_layer(64, 64, 2,
3   2, 32, first_block=True))
4 self.resnext_layer_2 = nn.Sequential(*self.resnext_layer(64, 128,
5   2, 2, 32))
6 self.resnext_layer_3 = nn.Sequential(*self.resnext_layer(128, 256,
7   2, 2, 32))
8 self.resnext_layer_4 = nn.Sequential(*self.resnext_layer(256, 512,
9   2, 2, 32))

```

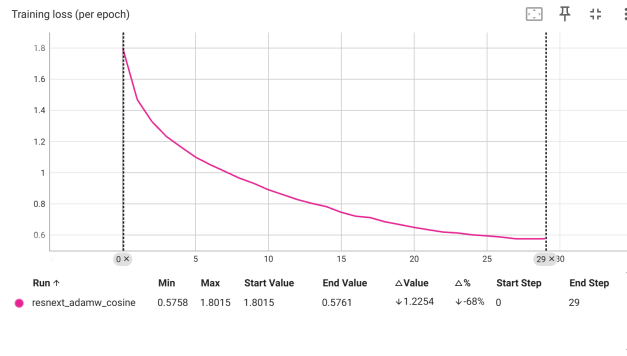
The configurations of our experiment on ResNext:

- optimizer: AdamW (AdamW_lr=0.001)
- scheduler: CosineAnnealingLR (CosineAnnealingLR_T_max=30)
- epoch_num=30
- batch_size=128

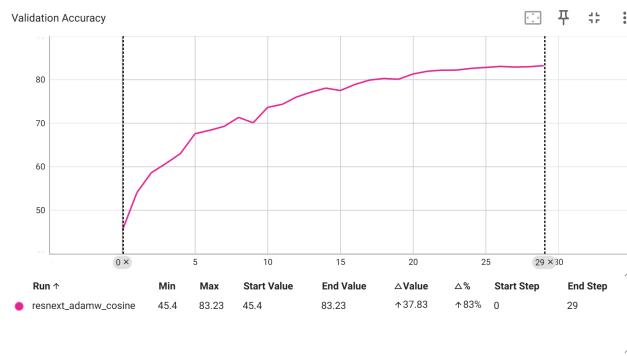
The test accuracy of our ResNeXt is 83.23%. It is still a success, although its accuracy turns out to be lower than VGG and ResNet of our own implementation. However, its effort to reduce FLOPs (equivalently, the amount of parameters) proves significant: the size of the model file it generates is less than one tenth that of VGG and ResNet!



(a) Training Accuracy



(b) Training Loss



(c) Validation Accuracy

Figure 4: ResNeXt

5 Train and test

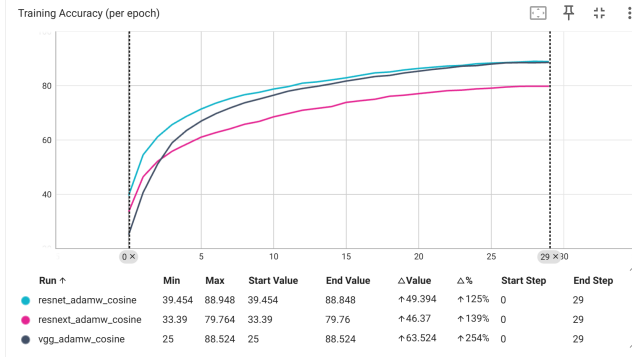
Please refer to **main.py** for our code in this part. It is almost the same as our train-and-test code in Homework 5.

We have made some minor adjustments so that our code can run on Google Colab's GPU. If you want to run these codes locally, just bear in mind that modifying file paths is necessary. For test on CPU devices, we have added a sentence to map the model file generated on GPU to the format that CPU can recognize and handle.

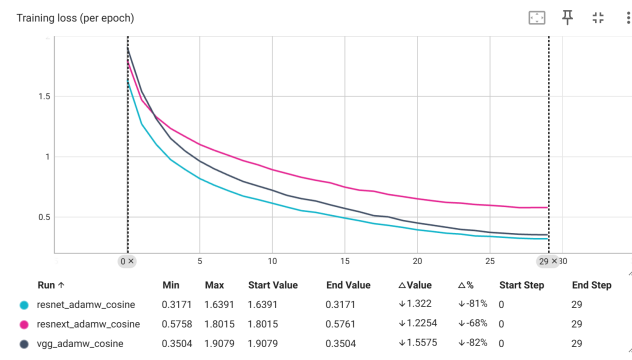
Since our GPU resources are limited, we use the test set as our validation set during the training process to avoid "surprise" after a long training period. (I **KNOW** that data should be segmented into training set, validation set and test set correctly!). Still, it is guaranteed that the training process will not come into contact with test data, so we hope this will not affect the rigor of our experiment.

You might have noticed that our accuracies on training data turn out to be lower than those on test data. This is a normal phenomena because we have used data augmentation to prevent overfitting, which may be a bit excessive; Yet, after all, it is not a bad thing.

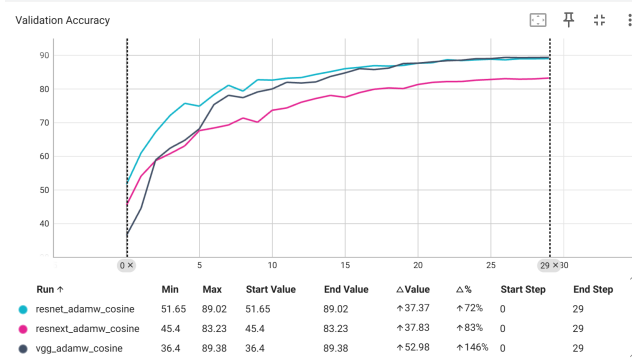
At the end of our report, it's time to take a family photo of our three convolutional neural network models! Curves from different models are put into the same figure so that we can see the differences in model performance more clearly.



(a) Training Accuracy



(b) Training Loss



(c) Validation Accuracy

Figure 5: Family Photo!